

Facultad de Ciencias de la Computación
Ingeniería en Software

**Análisis Comparativo de Arquitecturas
Distribuidas:**
Microservicios frente a Monolito Modular
**Aplicación al Sistema de Gestión Académica
de la Escuela “Provincias Unidas”**

TA-IND-01 — Informe Académico Individual

Asignatura: Aplicaciones Distribuidas — 7mo Nivel

Autor: Keyla Bedón Viteri

Docente: Ing. Gleiston Guerrero PhD.

Resumen

El presente informe analiza de forma comparativa dos estilos arquitectónicos ampliamente utilizados en el desarrollo de sistemas distribuidos: la arquitectura de microservicios y el monolito modular. Se examinan sus principios de diseño, ventajas, limitaciones y criterios de selección, tomando como referencia literatura académica reciente del ámbito de la ingeniería de software. El análisis se aplica al dominio del Sistema de Gestión Académica (SGA) de la Escuela “Provincias Unidas”, desarrollado como Proyecto Final de Curso (PFC), con el objetivo de justificar técnicamente la decisión arquitectónica adoptada. Los resultados evidencian que, para el contexto específico del SGA, la arquitectura de microservicios ofrece una mayor alineación con los principios de responsabilidad única y despliegue independiente, aunque introduce complejidad operacional que debe gestionarse con patrones adecuados.

Índice

1. Introducción	3
2. Marco Teórico	3
2.1. Arquitectura de Microservicios	3
2.2. Monolito Modular	4
2.3. Criterios de Comparación	4
3. Análisis Comparativo	5
3.1. Tabla Comparativa	5
3.2. Análisis Crítico	6
4. Aplicación al PFC: SGA Provincias Unidas	7
4.1. Contexto del Sistema	7
4.2. Justificación de la Arquitectura de Microservicios	7
4.3. Limitaciones Reconocidas y Mitigaciones	8
5. Conclusiones	8

1. Introducción

El diseño de la arquitectura de software constituye una de las decisiones más relevantes en el ciclo de vida de un sistema distribuido. La elección entre un estilo arquitectónico u otro tiene implicaciones directas sobre la escalabilidad, el mantenimiento, la velocidad de despliegue y la resiliencia del sistema ante fallos. En los últimos años, la arquitectura de microservicios ha ganado una adopción significativa en la industria; sin embargo, el monolito modular ha resurgido como una alternativa válida y, en ciertos contextos, superior.

El presente informe se enmarca en el desarrollo del Sistema de Gestión Académica (SGA) para la Escuela “Provincias Unidas”, institución educativa fiscal de nivel básico ubicada en Ecuador, que atiende a los subniveles Inicial, Preparatoria, Básica Elemental, Básica Media y Básica Superior. El sistema tiene como propósito digitalizar los procesos de matrículas, registro de calificaciones, control de asistencia y generación de reportes académicos conforme a la normativa del Ministerio de Educación (Mineduc).

Este informe aborda la siguiente pregunta de investigación: ¿Qué estilo arquitectónico microservicios o monolito modular resulta más adecuado para el desarrollo del SGA, considerando las restricciones técnicas, organizacionales y operacionales del entorno de despliegue?

El objetivo principal es realizar un análisis crítico comparativo que trascienda la simple descripción de ambas arquitecturas, y que permita fundamentar con argumentos técnicos la decisión adoptada en el PFC.

2. Marco Teórico

2.1. Arquitectura de Microservicios

La arquitectura de microservicios es un estilo de diseño en el que una aplicación se estructura como un conjunto de servicios pequeños, independientemente desplegables, organizados en torno a capacidades de negocio y comunicados a través de mecanismos ligeros, típicamente HTTP/REST o mensajería asíncrona. Este enfoque surgió como respuesta a las limitaciones de los sistemas monolíticos en entornos de alta demanda y ciclos de entrega continuos.

Cada microservicio sigue el Principio de Responsabilidad Única (SRP, por sus siglas en inglés): gestiona un dominio de negocio acotado y expone su funcionalidad a través de una interfaz bien definida. Esta característica favorece la evolución independiente de cada componente, el despliegue continuo (CI/CD) y la escalabilidad horizontal selectiva, permitiendo que únicamente el servicio con mayor carga se replique sin necesidad de escalar toda la aplicación.

Entre los patrones más relevantes de este estilo se encuentran: el API Gateway, como punto de entrada único que gestiona autenticación, enrutamiento y limitación de tasa;

el patrón de base de datos por servicio, que garantiza el desacoplamiento de datos entre servicios; y los patrones de resiliencia como Circuit Breaker y Retry, que mitigan el impacto de fallos en cascada.

Sin embargo, la adopción de microservicios no está exenta de desafíos. Soldani et al. [1] identifican como principales dificultades la gestión de la consistencia de datos distribuidos, la complejidad del monitoreo y la necesidad de infraestructura de orquestación madura. Estos autores documentan que organizaciones que adoptaron este estilo prematuramente enfrentaron incrementos significativos en el tiempo de desarrollo y en la tasa de incidentes operacionales.

2.2. Monolito Modular

El monolito modular es una variante del estilo monolítico tradicional en la que el código se organiza en módulos con fronteras bien definidas, interfaces explícitas y baja dependencia entre ellos, pero se despliega como una única unidad ejecutable [2]. A diferencia del monolito “gran bola de barro”, el monolito modular aplica principios de diseño limpio, cohesión alta y acoplamiento bajo al interior del mismo proceso.

Este estilo ha recuperado relevancia a partir de la constatación de que muchas organizaciones adoptaron microservicios prematuramente, asumiendo una complejidad operacional para la que no estaban preparadas [6]. El monolito modular permite mantener la simplicidad de despliegue y la trazabilidad directa del código, al tiempo que facilita una futura migración hacia microservicios si el crecimiento del sistema así lo requiere [2].

Desde una perspectiva de ingeniería, el monolito modular favorece las transacciones ACID nativas, la consistencia fuerte de datos y la depuración directa del sistema en un único proceso. Faustino et al. [2] demuestran empíricamente que la migración a un monolito modular ya introduce costos de rendimiento medibles, lo que cuestiona la asunción de que este paso intermedio es siempre trivial.

2.3. Criterios de Comparación

Para el análisis comparativo se emplean los siguientes criterios, derivados de la literatura revisada [1, 2, 5]:

- **Escalabilidad:** capacidad de escalar componentes de forma independiente ante incrementos de carga [3].
- **Despliegue independiente:** posibilidad de actualizar un componente sin afectar al resto del sistema [4].
- **Tolerancia a fallos:** comportamiento del sistema ante la caída de un componente [1].
- **Complejidad operacional:** esfuerzo requerido para operar, monitorear y mantener la infraestructura [5].

- **Consistencia de datos:** mecanismos para garantizar la integridad entre los datos gestionados por distintos componentes [5].
- **Velocidad de desarrollo inicial:** curva de aprendizaje y tiempo necesario para construir las primeras versiones funcionales [2].

3. Análisis Comparativo

3.1. Tabla Comparativa

Criterio	Microservicios	Monolito Modular
Escalabilidad [3]	Escalabilidad horizontal selectiva: se escala únicamente el servicio con mayor demanda sin replicar la aplicación completa.	Escalabilidad vertical o réplica completa. No permite escalar módulos individuales de forma independiente.
Despliegue independiente [4]	Cada servicio se despliega, actualiza y revierte de forma autónoma. Soporta CI/CD a nivel de componente.	Cualquier cambio requiere compilar y desplegar toda la aplicación. Ciclos de despliegue más largos.
Tolerancia a fallos [1]	Un servicio caído no detiene el sistema completo. Aplicación de patrones Circuit Breaker y fallbacks.	Un fallo no controlado puede propagar excepciones que afectan toda la aplicación.
Complejidad operacional [5]	Alta: requiere orquestación (Docker/Kubernetes), monitoreo distribuido, trazabilidad centralizada y gestión de configuración.	Baja: un único proceso, un único log, un único punto de despliegue. Operación directa y observable.
Consistencia de datos [5]	Consistencia eventual. Requiere patrones como Saga o eventos de dominio para operaciones distribuidas.	Transacciones ACID nativas. Consistencia fuerte sin esfuerzo adicional.

Criterio	Microservicios	Monolito Modular
Velocidad de desarrollo inicial [2]	Lenta: se requiere infraestructura de comunicación y contratos de API antes de entregar valor.	Rápida: única base de código, entorno local simple, sin dependencias entre equipos.

Cuadro 1: Comparación entre Microservicios y Monolito Modular

3.2. Análisis Crítico

La tabla 1 revela una tensión fundamental: los microservicios ofrecen ventajas estructurales en sistemas de gran escala y equipos distribuidos, pero transfieren complejidad desde el código hacia la infraestructura. El monolito modular, por el contrario, mantiene la simplicidad operacional a costa de limitar la autonomía de despliegue de cada componente [2].

Un aspecto frecuentemente omitido en comparaciones superficiales es que la mayor parte de los beneficios de los microservicios son relevantes a partir de cierto umbral de escala y madurez organizacional. Blinowski et al. [3] demuestran experimentalmente que en pruebas de concurrencia, el monolito puede superar al microservicio en hasta un 6% en rendimiento de throughput, debido a la ausencia de latencia de red entre módulos, aunque en escenarios de alta carga la diferencia se reduce considerablemente.

Desde una perspectiva crítica, el monolito modular no es simplemente un punto intermedio entre el monolito tradicional y los microservicios: es un estilo arquitectónico con mérito propio para sistemas de tamaño medio que no requieren despliegue independiente de sus partes. Soldani et al. [1] documentan que uno de los problemas recurrentes en migraciones a microservicios es la granularidad incorrecta de los servicios, generando el fenómeno denominado *distributed monolith*: un sistema con las desventajas de ambos estilos sin los beneficios de ninguno.

Bogner et al. [4] añaden que los *architecture smells* en sistemas basados en microservicios —como el acoplamiento temporal entre servicios o el antipatrón de servicio *chatty*— son más difíciles de detectar y corregir que sus equivalentes en un monolito, debido a la distribución de la lógica en múltiples repositorios y procesos. Cerny et al. [5] refuerzan esta observación mediante un estudio terciario que cataloga dichos antipatronos y propone técnicas de detección automatizada aplicables en entornos CI/CD.

Por tanto, la decisión arquitectónica no debe basarse en preferencias tecnológicas ni en tendencias de la industria, sino en un análisis riguroso de los atributos de calidad requeridos, las restricciones operacionales y la capacidad del equipo para gestionar la complejidad introducida [6].

4. Aplicación al PFC: SGA Provincias Unidas

4.1. Contexto del Sistema

El SGA de la Escuela “Provincias Unidas” es un sistema de información académica destinado a digitalizar la gestión de matrículas, calificaciones, asistencia, horarios y reportes de una institución con aproximadamente 200 estudiantes y 14 docentes. El sistema será utilizado por cuatro perfiles de usuario: Director, Secretaría, Docente y Soporte Técnico. El entorno de despliegue es un servidor local con capacidad limitada, gestionado por personal con conocimientos técnicos básicos.

La arquitectura implementada se compone de dos microservicios: el **MS Principal** (Java, Spring Boot, puerto 8080), responsable de la gestión administrativa, y el **MS Docente** (Python, FastAPI, puerto 8081), responsable de la gestión pedagógica. Ambos se comunican a través de un API Gateway (Spring Cloud Gateway) y RabbitMQ para eventos asíncronos.

4.2. Justificación de la Arquitectura de Microservicios

A pesar de las advertencias sobre la complejidad operacional de los microservicios en contextos de pequeña escala [1], la decisión de adoptar este estilo en el SGA se fundamenta en los siguientes argumentos técnicos:

Separación clara de dominios de negocio. El sistema presenta dos dominios funcionalmente distintos: la gestión administrativa (matrículas, estudiantes, usuarios, grados, años lectivos) y la gestión pedagógica (calificaciones, asistencia, horarios, escalas de evaluación). Estos dominios tienen ciclos de cambio diferenciados: la lógica de calificaciones está sujeta a actualizaciones frecuentes por cambios normativos del Mineduc, mientras que la gestión administrativa es más estable. Desacoplar ambos dominios en servicios independientes permite actualizar el módulo docente sin afectar la disponibilidad del módulo administrativo, en línea con los principios de evolución independiente documentados por Bogner et al. [4].

Heterogeneidad tecnológica justificada. El MS Principal se implementa en Java con Spring Boot, aprovechando la robustez de Spring Security para la gestión de autenticación y autorización basada en roles JWT. El MS Docente se implementa en Python con FastAPI, que ofrece mayor expresividad para la lógica de cálculo de promedios, conversión de escalas y generación de reportes con ReportLab. Esta heterogeneidad tecnológica es una característica definitoria de la arquitectura de microservicios y no sería viable en un monolito modular sin introducir complejidad estructural significativa [4].

Tolerancia a fallos operacionalmente relevante. En el contexto escolar, la disponibilidad del módulo de calificaciones durante el período de evaluaciones es crítica. Con una arquitectura de microservicios, una caída del módulo administrativo no impide que

los docentes registren calificaciones o consulten horarios. En un monolito, cualquier fallo no controlado en el módulo de matrículas podría propagarse y detener toda la aplicación, afectando a los 14 docentes simultáneamente [1].

Gestión asíncrona de eventos mediante RabbitMQ. La propagación de eventos entre microservicios —como la creación de un año lectivo o la asignación de horarios se implementa con RabbitMQ en modo asíncrono. Esto permite que el Director complete la operación administrativa sin esperar a que el módulo docente procese el evento, mejorando la experiencia de usuario y desacoplando el comportamiento de ambos servicios [5].

4.3. Limitaciones Reconocidas y Mitigaciones

La adopción de microservicios en un equipo de cuatro integrantes introduce una complejidad operacional que debe gestionarse conscientemente [1]. Las principales limitaciones identificadas y sus mitigaciones son:

- **Complejidad de despliegue:** mitigada mediante Docker Compose, que orquesta los contenedores (gateway, ms-principal, ms-docente, postgres, rabbitmq, frontend) con un único comando [4].
- **Consistencia eventual:** mitigada acotando las operaciones distribuidas a eventos de bajo riesgo (propagación de configuración), mientras las operaciones críticas (matrículas, calificaciones) se resuelven dentro de su microservicio con transacciones ACID locales [5].
- **Trazabilidad:** mitigada con logging centralizado mediante identificadores de correlación en las cabeceras HTTP propagados por el API Gateway [6].

5. Conclusiones

El análisis comparativo entre microservicios y monolito modular evidencia que no existe una arquitectura universalmente superior: la decisión correcta depende del contexto específico del sistema, del equipo y del entorno operacional [3]. El monolito modular es una opción válida y recomendable para equipos pequeños con dominios homogéneos y sin requisitos diferenciados de escalabilidad o despliegue [2].

Sin embargo, en el caso del SGA de la Escuela “Provincias Unidas”, la separación funcional entre el dominio administrativo y el pedagógico, la heterogeneidad tecnológica entre Java y Python, y la necesidad de actualizar el módulo de calificaciones de forma independiente por cambios normativos del Mineduc, justifican la adopción de la arquitectura de microservicios [5]. Las limitaciones operacionales se mitigan con Docker Compose y patrones de diseño adecuados, manteniendo la complejidad dentro de un nivel manejable [1].

Un aprendizaje crítico de este análisis es que la arquitectura de microservicios debe adoptarse por razones técnicas verificables, no por tendencia tecnológica [6]. La pregunta pertinente no es “¿microservicios o monolito?”, sino “¿qué atributos de calidad son no negociables para este sistema en este contexto?”. Responder esa pregunta con rigor es la base de una decisión arquitectónica defendible [4].

Referencias

Referencias

- [1] J. Soldani, D. A. Tamburri, and W.-J. van den Heuvel, “The pains and gains of microservices: A systematic grey literature review,” *Journal of Systems and Software*, vol. 146, pp. 215–232, 2018. doi: 10.1016/j.jss.2018.09.082
- [2] D. Faustino, N. Gonçalves, M. Portela, and A. R. Silva, “Stepwise migration of a monolith to a microservices architecture: Performance and migration effort evaluation,” *arXiv preprint arXiv:2201.07226*, 2022. doi: 10.48550/arXiv.2201.07226
- [3] G. Blinowski, A. Ojdowska, and A. Przybyłek, “Monolithic vs. microservice architecture: A performance and scalability evaluation,” *IEEE Access*, vol. 10, pp. 20357–20374, 2022. doi: 10.1109/ACCESS.2022.3152803
- [4] J. Bogner, J. Fritsch, S. Wagner, and A. Zimmermann, “Industry practices and challenges for the evolvability assurance of microservices,” *Empirical Software Engineering*, vol. 26, no. 5, p. 104, 2021. doi: 10.1007/s10664-021-09999-9
- [5] T. Cerny, A. S. Abdelfattah, A. Al Maruf, A. Janes, and D. Taibi, “Catalog and detection techniques of microservice anti-patterns and bad smells: A tertiary study,” *Journal of Systems and Software*, vol. 206, p. 111829, 2023. doi: 10.1016/j.jss.2023.111829
- [6] I. Oumoussa and R. Saidi, “Evolution of microservices identification in monolith decomposition: A systematic review,” *IEEE Access*, vol. 12, pp. 23389–23405, 2024. doi: 10.1109/ACCESS.2024.3365079